



MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN SUPERIOR
UNIVERSIDAD SIMÓN BOLÍVAR
DEPARTAMENTO DE COMPUTACIÓN Y CIENCIAS DE LA INFORMACIÓN
TALLER DE BASES DE DATOS I - CI-3391
SARTENEJAS, DISTRITO CAPITAL

Informe Técnico — Corte 4

Docente: Ing. Juan Andres Cuevas

Estudiantes: Cristina Puyosa 23-10395

Alejandro Villamizar 22-10439

Carlos Bisogno 22-10041

Sartenejas, septiembre 2026

Informe Técnico — Corte 4

Decisiones de diseño generales

Antes de entrar en el detalle de cada endpoint, conviene explicar algunas decisiones que se repiten a lo largo de todo el proyecto, porque entenderlas de una vez evita repetir la misma justificación en cada sección.

La conexión a la base de datos se abre y se cierra dentro de cada función de endpoint, mediante la función auxiliar `get_conexion()`. Se evaluó usar un pool de conexiones (por ejemplo `psycopg2.pool.SimpleConnectionPool`), que en un entorno de producción sería preferible porque reutiliza conexiones en vez de crear una nueva por cada petición HTTP. Para el volumen de tráfico que maneja este proyecto académico, sin embargo, el costo de abrir una conexión por request es despreciable, y el patrón de `try / finally` con cierre explícito es más simple de leer y de depurar. Se documenta esta limitación en un sistema real de videovigilancia con múltiples cámaras enviando eventos de forma constante, por ende un pool sería la elección correcta.

Todas las consultas que reciben datos del cliente los pasan como parámetros (%) en vez de concatenarlos directamente en el texto SQL. Esto es lo que evita la inyección SQL. El único lugar donde el código construye una parte del SQL de forma dinámica es en los endpoints PUT, donde la cláusula SET se arma según qué campos envió el cliente. Ahí es importante notar que lo dinámico son los *nombres de columna*, que provienen de un diccionario fijo definido en el propio código (nunca de lo que escribe el usuario), mientras que los *valores* siguen viajando parametrizados. Esa separación es la que hace seguro el patrón, y es distinta de construir el SQL completo a partir de texto arbitrario del cliente.

El manejo de errores sigue una jerarquía: primero se capturan las excepciones específicas de PostgreSQL que tienen un significado claro para el cliente de la API (como una violación de llave foránea), y solo después un `except psycopg2.Error` genérico que cubre cualquier otro fallo de base de datos con un 500. Esta jerarquía importa porque en Python el primer bloque `except` que coincide es el que se ejecuta, así que las excepciones más específicas siempre van antes que las genéricas. El bloque `try` que envuelve cada endpoint también cubre los problemas de conexión con el servidor de base de datos: si Postgres no está disponible, o la conexión se cae a mitad de una operación, `psycopg2` lanza una excepción que igualmente cae dentro de `except psycopg2.Error`, y el cliente recibe un 500 explicativo en vez de que el proceso de FastAPI se caiga sin dar ninguna respuesta.

Todos los endpoints que devuelven datos usan `cursor_factory=RealDictCursor` en vez del cursor por defecto de `psycopg2`. Por defecto, `psycopg2` devuelve cada fila como una tupla indexada por posición (`fila[0]`, `fila[1]`, etc.), lo que obliga a recordar el orden exacto de las columnas de la consulta y hace el código frágil ante cualquier cambio en el SELECT. `RealDictCursor` en cambio mapea cada columna a su nombre real dentro de un diccionario, lo que hace el código más legible y menos propenso a errores. Esa elección tiene además una ventaja directa para una API REST: al hacer `return` de ese diccionario (o de una lista de diccionarios) desde una función de FastAPI, el framework lo serializa automáticamente a JSON, que es el formato en el que normalmente responden las APIs, sin que el propio endpoint tenga que construir esa conversión a mano.

Los modelos de Pydantic (`Ubicacion`, `Camara`, `Evento`, `BusquedaVector`, y sus variantes `*Update`) cumplen tres funciones a la vez: definen la estructura esperada de cada entidad mediante clases y anotaciones de tipo, validan automáticamente los datos que llegan en cada petición (por ejemplo, que `latitude` sea un número y no un texto, o que el vector de búsqueda tenga exactamente 512 elementos), y

transforman esos datos ya validados en objetos Python listos para usar dentro del endpoint. Gracias a esto, buena parte de los errores de formato nunca llegan a ejecutar una sola línea de SQL: FastAPI responde 422 antes de que el endpoint se ejecute.

Por último, la contraseña y demás datos de conexión a la base no están escritos en el código fuente, sino que se leen desde variables de entorno con la librería `python-dotenv`. Esta decisión y su justificación se explican con más detalle en la sección 6, junto con el manejo del archivo `.gitignore`.

Entregable 4A: Endpoints CRUD

Ubicaciones

`GET /ubicaciones` devuelve todas las filas de la tabla `location`. Se usa `RealDictCursor` en vez del cursor por defecto de `psycopg2` porque este devuelve cada fila como un diccionario con los nombres de columna como llaves, y FastAPI puede serializar ese diccionario directamente a JSON sin que haya que transformarlo a mano.

`POST /ubicaciones` inserta una nueva ubicación. El modelo `Ubicacion` de Pydantic no incluye el campo `uid`, porque esa clave la genera la propia base de datos (se asume un valor por defecto en la columna, como `gen_random_uuid()`). Pedirle al cliente que invente su propia clave primaria sería delegarle una responsabilidad que le corresponde a la base de datos, y abriría la puerta a colisiones si dos clientes generan el mismo UUID por error. El `INSERT` usa `RETURNING` para devolver la fila completa recién creada, incluyendo el `uid` generado, de modo que el cliente no tiene que hacer un segundo `GET` para saber qué UID le tocó a su nuevo recurso.

`PUT /ubicaciones/{id}` implementa una actualización parcial: el modelo `UbicacionUpdate` tiene todos sus campos opcionales, y el endpoint arma un diccionario solo con los campos que el cliente efectivamente envió, descartando los que llegaron como `None`. Se eligió actualización parcial en vez de reemplazo total porque es el comportamiento más natural para un cliente que solo quiere corregir, por ejemplo, el piso de una ubicación sin tener que reenviar el nombre, el tipo de zona y las coordenadas que no cambiaron. Si no se envía ningún campo, el endpoint responde `400` en vez de ejecutar un `UPDATE` vacío o sin sentido. Si el `uid` no corresponde a ninguna fila, `RETURNING` no devuelve nada y el endpoint responde `404`.

`DELETE /ubicaciones/{id}` elimina la ubicación indicada. Aquí se decidió capturar específicamente `psycopg2.errors.ForeignKeyViolation`: si la ubicación tiene cámaras asociadas, la base de datos rechaza el `DELETE` por la llave foránea, y ese caso se traduce a un `409 Conflict` en vez de un `500`. La diferencia importa porque un `500` le dice al cliente "el servidor falló", mientras que un `409` le dice "tu petición es válida pero choca con el estado actual del sistema" — que es justamente lo que está pasando: no hay ningún error de programación, simplemente no se puede borrar una ubicación mientras siga teniendo cámaras.

Cámaras

`GET /camaras` y `GET /camaras/{id}` siguen el mismo patrón que ubicaciones: listar todas las filas o buscar una por su clave, devolviendo `404` si no existe.

`POST /camaras` inserta una cámara nueva. A diferencia de `Ubicacion`, el modelo `Camara` sí incluye un campo (UID) que el cliente debe enviar, pero en este caso no es la clave primaria de la cámara sino la llave foránea hacia la ubicación a la que pertenece: la propia clave de la cámara (`cid`) la sigue generando la base de datos y se recupera con `RETURNING`. Ese UID obligatorio existe porque conceptualmente no hay forma de crear una cámara sin decir a qué ubicación pertenece.

PUT /camaras/{id} funciona igual que el de ubicaciones: actualización parcial, con la misma lógica de descartar campos no enviados y devolver 404 si el cid no existe.

Eventos

GET /eventos lista todos los eventos registrados. POST /eventos inserta un evento nuevo asociado a una cámara (cid), con su nivel de confianza y la posición del bounding box detectado (posx, posy, width, height), el timestamp (etime) lo asigna la base de datos por defecto, no el cliente, para que refleje el momento real de inserción del evento y no un valor que el cliente podría manipular. GET /eventos/{id} busca un evento puntual y responde 404 si no existe. A diferencia de ubicaciones y cámaras, eventos no tiene PUT ni DELETE: un evento representa algo que efectivamente ocurrió, y editar o borrar ese registro rompería la trazabilidad histórica del sistema, que es justamente el propósito de un sistema de videovigilancia.

Entregable 4B

GET /analytics/zones/{tipo} invoca la función PL/pgSQL get_zone_summary(). Antes de llamarla, el endpoint valida que el tipo recibido corresponda a un zone_type que realmente exista en la tabla location, si no, responde 404 en vez de dejar que la función devuelva una tabla vacía sin explicación. Esta validación se hace con una consulta simple de tipos distintos, cerrando ese cursor antes de abrir uno nuevo para la consulta principal, de modo que no queden cursores abiertos innecesariamente sobre la misma conexión.

GET /analytics/cameras/{id}/traffic invoca get_camera_traffic(), recibiendo un rango de fechas a través de los query parameters from y to. Como from es una palabra reservada en Python, no se puede usar directamente como nombre de parámetro de la función, se resuelve con Query(..., alias="from"), que le permite al parámetro interno llamarse desde mientras la URL sigue usando from tal como pide el enunciado. Los parámetros se tipan como date, de modo que FastAPI valida automáticamente el formato de fecha y responde 422 si el cliente envía algo que no se pueda interpretar como fecha, sin que el propio endpoint tenga que parsear el texto a mano.

GET /analytics/alerts/summary calcula, para cada cámara, el conteo de alertas por severidad (crítica, alta, media, baja) dentro de una ventana de días configurable con el query parameter days (por defecto 30). La consulta usa LEFT JOIN desde camera hacia event y alert, de modo que las cámaras sin alertas en el período aparecen igual en el resultado con conteos en cero, en vez de desaparecer de la respuesta como pasaría con un INNER JOIN. El filtro de fecha se aplica dentro de la condición del JOIN con alert (no en un WHERE posterior), precisamente para no descartar filas de cámaras que no tuvieron ninguna alerta reciente.

Entregable 4C: Endpoints vectoriales

GET /objects/{id}/similar primero verifica que el objeto de referencia exista en la tabla object, si no, responde 404 antes de siquiera intentar la búsqueda de similitud. Luego invoca find_similar_objects(), que internamente ya se encarga de excluir al propio objeto de referencia de los resultados y de filtrar por el mismo tipo de objeto (persona o vehículo, según corresponda). Los parámetros threshold y limit llegan como query parameters con valores por defecto (0.20 y 5 respectivamente), tal como pide el enunciado, y la respuesta se arma como un diccionario JSON con la lista de objetos similares junto a su distancia coseno y el contexto de detección (cámara y momento).

POST /search/similar es el endpoint que simula la llegada de un embedding externo, por ejemplo el vector calculado por un modelo de visión por computador a partir de una imagen capturada en

campo. El modelo `BusquedaVector` de `Pydantic` exige que el vector tenga exactamente 512 elementos (`min_length=512`, `max_length=512`), lo que rechaza automáticamente con 422 cualquier vector de tamaño distinto sin que el endpoint tenga que validarlo a mano. Como `psycpg2` no sabe convertir una lista de Python al tipo vector de `pgvector`, la función auxiliar `vector_a_literal()` transforma la lista en el literal de texto que `pgvector` sí entiende (`[0.1,0.2,...]`), y ese texto se castea explícitamente a vector dentro del propio SQL con `%s::vector`. La búsqueda usa el operador de distancia coseno de `pgvector` (`<=>`) tanto en el `SELECT` como en el `ORDER BY`, y ordena de menor a mayor distancia para que los resultados más parecidos aparezcan primero. El filtro opcional por tipo (persona o vehículo) se agrega al `WHERE` solo si el cliente lo especifica, manteniendo el mismo cuidado de pasar el valor como parámetro y no concatenado.

Configuración y control de versiones

Un aspecto que no es un endpoint pero forma parte de las decisiones de diseño del proyecto es cómo se maneja la configuración sensible. En una primera versión del código, la contraseña de la base de datos estaba escrita directamente en el archivo fuente. Esto se corrigió moviendo todos los datos de conexión (nombre de la base, usuario, contraseña, host y puerto) a variables de entorno, leídas con la librería `python-dotenv` desde un archivo `.env` que vive únicamente en la máquina de cada desarrollador y nunca se sube al repositorio.

Para que el repositorio siga siendo reproducible por cualquier otra persona sin exponer la contraseña real, se agregó un archivo `.env.example`: una plantilla con los mismos nombres de variable pero sin valores sensibles, que cualquiera puede copiar como `.env` y completar con sus propios datos de conexión. Este archivo sí se sube al repositorio, precisamente porque no contiene ningún secreto.

El archivo `.gitignore` es el mecanismo que impide que Git incluya ciertos archivos en los commits. Se agregó una entrada para `.env`, de modo que aunque el archivo exista en la carpeta del proyecto, Git lo ignora y nunca queda registrado en el historial del repositorio. También se agregaron entradas para `__pycache__` y los entornos virtuales, que son archivos generados automáticamente por Python y no aportan nada al código fuente versionado. Esta separación entre lo que se comparte (`.env.example`, el código) y lo que se queda local (`.env`) es la práctica estándar para manejar credenciales en cualquier proyecto que use control de versiones.